

Project Proposal of Splendor

1. Project Overview

Provide a brief overview of your project, explaining the game's concept, mechanics, and functionality.

Concept

*This project is a digital implementation of the popular engine-building board game, **Splendor**, developed using the **Pygame** library. The game challenges players to take on the role of wealthy Renaissance merchants, strategically collecting gemstones to acquire development cards, gain prestige points, and attract the attention of noble patrons.*

Game Mechanics

The core gameplay follows the official Splendor rules:

- **Resource Management:** *Players collect five types of gem tokens (Emerald, Sapphire, Ruby, Diamond, and Onyx) plus wild Gold tokens.*
- **Engine Building:** *Players spend tokens to purchase **Development Cards**, which provide permanent gem bonuses for future purchases and **Prestige Points**.*
- **Victory Conditions:** *The first player to reach **15 Prestige Points** triggers the final round. The player with the highest score wins.*
- **AI Opponent:** *The game features a single-player mode where the user competes against a rule-based **Bot** programmed with heuristic strategic decision-making.*

Functionality & Technical Highlights

- **Object-Oriented Design:** *Developed using a strict **OOP structure** with classes for the Game Engine, Players, Cards, and UI elements.*

- **Interactive UI:** A fully functional graphical user interface (GUI) built with Pygame for seamless mouse interaction and real-time visual updates.
- **Data Collection & Analytics:** The system automatically tracks gameplay metrics—such as average turns per win, gem selection frequency, and card-tier efficiency.
- **Data Visualization:** Includes a dedicated statistics module to visualize collected data through charts, providing insights into game balance and AI performance.

2. Project Review

Study and then present a relevant existing project, outlining the specific improvements you plan to make. Highlight the key areas where your project enhances or builds upon the original.

Reference Project

Most open-source **Splendor-Pygame** implementations found on platforms like GitHub focus primarily on basic game mechanics, such as UI rendering and local multiplayer logic. These versions typically serve as simple digital clones of the board game without advanced backend features.

Limitations of Existing Projects

- **Lack of Automation:** Most require two human players; they lack a functional AI opponent.
- **No Data Persistence:** Gameplay data is lost once the window is closed; there is no tracking of player behavior or game balance.
- **Basic Architecture:** Code is often written in a single script or lacks a clear **Object-Oriented Programming (OOP)** hierarchy, making it difficult to scale.

Key Enhancements in This Project

1. ***Heuristic AI Opponent (Single-player Mode):*** Unlike basic versions, this project implements a **Rule-based Bot**. It doesn't just play randomly but evaluates the "Efficiency Score" of cards to simulate a challenging human opponent.
2. ***Integrated Data Analytics Suite:*** A major upgrade is the **Logging System**. The game captures over 100 records of specific features (e.g., gem priority, turn-to-win ratio). This transforms the game from a simple toy into a data-driven tool for strategy analysis.
3. ***Advanced OOP & Modular Design:*** This project strictly follows **OOP principles** with at least 5 distinct classes (Player, Card, Board, Bank, and StatsManager). This ensures clean code, better bug management, and a professional software structure.
4. ***Statistical Visualization:*** While original projects only show the game screen, this version includes a **Visualization Module** to generate charts (via Matplotlib or internal UI), providing meaningful insights into gameplay patterns.

3. Programming Development

3.1 Game Concept

Describe the game in detail, including its mechanics and objectives. Highlight the key features or selling points of the game.

***Splendor (Pygame Edition)** is a strategic engine-building game where players act as Renaissance merchants.*

- **Objectives:** Collect gemstone tokens to purchase **Development Cards**. These cards grant permanent gem discounts and **Prestige Points**. The first player to reach **15 points** wins.
- **Mechanics:** On each turn, a player can: (1) Take 3 different gems, (2) Take 2 gems of the same color, (3) Reserve a card, or (4) Purchase a card.
- **Key Selling Points:**

***Intelligent Heuristic AI:** A challenging single-player experience featuring a rule-based bot that evaluates the "Efficiency Score" of each move, simulating a competitive human opponent.*

***Post-Game Strategy Analytics:** A dedicated statistics module that processes gameplay records to visualize player behavior, win-rate trends, and resource management efficiency through graphical charts.*

3.2 Object-Oriented Programming Implementation

List and describe **at least five classes** you will implement in your game. For each class, briefly explain its role, key attributes, and methods. You can also include a UML diagram for better understanding. (Later, this UML will be required as part of the full proposal.)

1) Card System

1.1 Card Class

Role:

Represents a single development card in the game.

Responsibility:

This class stores all essential information about one card, including its purchase cost, prestige value, permanent gem bonus, and tier level.

Key Attributes:

- *cost (dict): Gem requirements needed to buy the card*
- *points (int): Prestige points earned after purchase*
- *gem_bonus (str): Permanent gem discount granted by the card*
- *tier (int): Card level (Tier 1, Tier 2, or Tier 3)*
- *card_id (int or str): Unique identifier for the card*

Key Methods:

- *can_afford(player) → Checks whether the player has enough tokens and bonuses*
- *get_display_data() → Returns formatted card data for UI rendering*

1.2 Deck Class

Role:

Manages a collection of cards in a specific tier.

Responsibility:

Instead of storing all cards directly inside the game engine, each deck handles its own card pool. This makes shuffling, drawing, and refilling board cards much cleaner.

Key Attributes:

- *cards (list): List of hidden cards in the deck*
- *tier (int): Deck level (1, 2, or 3)*

Key Methods:

- *shuffle()* → *Randomizes card order*
- *draw()* → *Removes and returns the top card*
- *is_empty()* → *Checks whether the deck still contains cards*

1.3 BoardCardManager Class

Role:

Controls the face-up development cards visible on the board.

Responsibility:

Splendor requires four open cards from each tier to be available at all times when possible. This class keeps those visible cards updated.

Key Attributes:

- *visible_cards (dict): Open cards grouped by tier*
- *decks (dict): References to the three card decks*

Key Methods:

- *setup_board()* → *Draws initial face-up cards*
- *remove_card(card)* → *Removes a purchased/reserved card*
- *refill_card(tier)* → *Draws a replacement card from the correct deck*
- *get_all_visible_cards()* → *Returns all current board cards*

2) Noble System

2.1 Noble Class

Role:

Represents a noble tile in the game.

Responsibility:

A noble has no purchase cost in tokens, but is awarded automatically when a player reaches the required permanent bonuses.

Key Attributes:

- *requirements (dict): Required permanent bonuses by color*
- *points (int): Prestige points given by the noble*
- *noble_id (int or str): Unique noble identifier*

Key Methods:

- *can_visit(player)* → *Checks whether the player qualifies for the noble*

2.2 NobleManager Class

Role:

Handles all nobles currently active in the match.

Responsibility:

At the beginning of the game, only a subset of nobles is used. This class manages those active nobles and checks whether a player can claim one after their turn.

Key Attributes:

- *active_nobles (list): Nobles currently available in the match*

Key Methods:

- *setup_nobles(player_count) → Selects nobles based on game setup*
- *check_noble_visit(player) → Determines whether the player earns a noble*
- *remove_noble(noble) → Removes a claimed noble from play*

3) Token and Resource System

3.1 Bank Class

Role:

Represents the central token supply of the game.

Responsibility:

The bank stores all gem tokens and gold tokens and enforces availability rules when players take or return tokens.

Key Attributes:

- *tokens (dict): Number of available tokens by color*
- *gold_tokens (int): Number of gold tokens remaining*

Key Methods:

- *can_take(selection) → Validates whether the chosen token action is legal*
- *take_tokens(selection) → Deducts selected tokens from the bank*
- *return_tokens(selection) → Adds returned tokens back to the bank*
- *take_gold() → Gives a gold token if available*

3.2 CostCalculator Class

Role:

Calculates how much a player actually needs to pay for a card.

Responsibility:

In Splendor, the real payment depends on both tokens and permanent discounts. This logic is important enough to be isolated instead of buried inside Player or Card.

Key Attributes:

- *None required, or utility-based class*

Key Methods:

- *calculate_discounted_cost(card, player) → Applies player bonuses to card cost*
- *calculate_gold_needed(card, player) → Determines how many gold tokens are needed*
- *can_complete_purchase(card, player) → Final affordability check*

4) Player System

4.1 Player Class

Role:

Represents a human player's state and actions.

Responsibility:

Stores all resources and game progress associated with one player.

Key Attributes:

- *name (str): Player name*
- *wallet (dict): Current token counts*
- *bonuses (dict): Permanent gem discounts from purchased cards*
- *reserved_cards (list): Reserved cards held by the player*
- *owned_cards (list): Purchased development cards*
- *owned_nobles (list): Claimed nobles*
- *score (int): Total prestige points*

Key Methods:

- *collect_gems(selection) → Adds tokens to the player wallet*
- *purchase_card(card) → Pays for and acquires a card*
- *reserve_card(card) → Reserves a card for later*
- *add_noble(noble) → Adds a noble and updates score*
- *must_return_tokens() → Checks whether token limit is exceeded*
- *return_excess_tokens(selection) → Returns tokens to the bank if over limit*

4.2 BotPlayer Class

Role:

Represents the AI-controlled opponent.

Responsibility:

Extends the normal player behavior with automated decision-making.

Key Attributes:

- *Inherits all Player attributes*
- *target_card (optional): Current strategic target*
- *difficulty (str or int): AI level or behavior profile*

Key Methods:

- *choose_action(game_state) → Selects the best move*
- *evaluate_card_priority(card) → Scores card usefulness*
- *choose_gems(game_state) → Chooses gem tokens strategically*
- *decide_reservation(game_state) → Determines whether to reserve a card*

5) Game Flow System

5.1 GameEngine Class

Role:

Acts as the main controller of the entire game.

Responsibility:

Coordinates all game systems and serves as the central point of control. However, it should not do everything alone; instead, it communicates with specialized classes.

Key Attributes:

- *players (list): All players in the game*
- *current_turn (int): Index of active player*
- *is_over (bool): Indicates whether the game has ended*
- *winner (Player or None): Winning player when the game ends*

Key Methods:

- *start_game() → Initializes the match*
- *process_action(action) → Sends the chosen action through the rule system*
- *end_turn() → Moves to the next player*
- *check_endgame() → Determines whether the final round should begin*
- *determine_winner() → Finds the winner after game end*

5.2 TurnManager Class

Role:

Handles turn order and round progression.

Responsibility:

Instead of putting all turn logic inside GameEngine, this class focuses only on whose turn it is and how turn progression works.

Key Attributes:

- *turn_index (int): Current active player*
- *round_number (int): Current round*
- *final_round_triggered (bool): Indicates whether endgame has started*

Key Methods:

- *next_player(player_count) → Advances turn to the next player*
- *start_final_round() → Marks the final cycle of turns*
- *is_last_turn() → Checks whether the game should stop after this round*

5.3 RuleManager Class

Role:

Validates actions according to Splendor rules.

Responsibility:

All rules—such as legal gem collection, reservation limits, purchase validation, and token limits—should be checked here rather than spread throughout the codebase.

Key Attributes:

- *None required, or references to bank/game state*

Key Methods:

- `validate_gem_collection(player, selection, bank)`
- `validate_card_purchase(player, card)`
- `validate_reservation(player, card)`
- `validate_token_limit(player)`
- `validate_action(player, action, game_state)`

5.4 ActionHandler Class

Role:

Executes validated player actions.

Responsibility:

Once the rule manager approves an action, this class performs the actual state changes.

Key Attributes:

- *Reference to GameEngine or game state*

Key Methods:

- `handle_take_tokens(player, selection)`
- `handle_purchase(player, card)`
- `handle_reserve(player, card)`
- `handle_return_tokens(player, selection)`

6) User Interface System

6.1 GameUI Class

Role:

Main visual controller for the Pygame interface.

Responsibility:

Handles screen rendering, layout updates, and coordination between the backend logic and the visual layer.

Key Attributes:

- *screen (Pygame surface): Main display surface*
- *font_manager (object): Text rendering helper*
- *selected_object (object): Current selected item*
- *buttons (list): Interactive UI elements*

Key Methods:

- *draw_game_state(game_state) → Renders the current board*
- *update_display() → Refreshes the screen*
- *show_message(text) → Displays notifications or errors*

6.2 Button Class

Role:

Represents clickable UI buttons.

Responsibility:

Reusable component for actions such as End Turn, Reserve, Buy, Confirm, or View Stats.

Key Attributes:

- *rect*
- *label*

- `action_id`
- `is_active`

Key Methods:

- `draw(screen)`
- `is_clicked(mouse_pos)`

6.3 CardSprite Class

Role:

Represents the visual display of a card.

Responsibility:

Separates the logical Card data from the way it is drawn on screen.

Key Attributes:

- `card`
- `rect`
- `is_highlighted`

Key Methods:

- `draw(screen)`
- `contains_point(pos)`
- `highlight()`

6.4 TokenSprite Class

Role:

Represents the visual display of a gem token.

Responsibility:

Used for rendering token stacks and allowing mouse interaction.

Key Attributes:

- `color`
- `position`
- `count`

Key Methods:

- `draw(screen)`
- `update_count(new_count)`

6.5 InputHandler Class

Role:

Processes mouse and keyboard input.

Responsibility:

Translates user interaction into game actions.

Key Attributes:

- `current_selection`
- `hovered_object`

Key Methods:

- `handle_mouse_click(pos)`
- `handle_hover(pos)`

- `translate_to_action()`

7) AI Decision System

7.1 BotStrategy Class

Role:

Contains the decision-making logic of the bot.

Responsibility:

Rather than putting all AI logic directly into BotPlayer, this class allows the behavior model to be designed and improved independently.

Key Attributes:

- `priority_weights` (dict): Scoring weights for AI evaluation
- `strategy_mode` (str): Aggressive, balanced, defensive, etc.

Key Methods:

- `select_best_action(bot, game_state)`
 - `score_card(card, bot)`
 - `score_token_choice(choice, bot)`
 - `choose_target_card(bot, game_state)`
-

8) Data Collection and Analytics System

8.1 DataLogger Class

Role:

Records gameplay events during each match.

Responsibility:

Stores actions and game outcomes into a persistent file format such as CSV or JSON.

Key Attributes:

- *session_data (list)*
- *file_path (str)*
- *record_count (int)*

Key Methods:

- *log_move(player, action)*
 - *log_turn_summary(turn_data)*
 - *log_game_result(result)*
 - *save_session()*
-

8.2 AnalyticsEngine Class

Role:

Analyzes stored gameplay data.

Responsibility:

Transforms raw records into meaningful statistics for evaluating game balance and bot performance.

Key Attributes:

- *metrics_summary (dict)*

- *history_buffer* (list)
- *plot_style* (str)

Key Methods:

- *calculate_win_rate()*
- *calculate_average_turns()*
- *analyze_gem_frequency()*
- *analyze_tier_efficiency()*
- *render_charts()*

⚠ Note : AnalyticsEngine Class just is idea , Now i dont use it in practical yet.

I dont sure it will work.

3.3 Algorithms Involved

Mention the algorithms used in the game. If your game incorporates techniques such as pathfinding, sorting, AI behavior, rule-based logic, or event-driven mechanics, describe them here.

- *The bot in this project uses a **heuristic-based decision system**, which is a rule-based approach rather than a learning AI model. At each turn, it evaluates all possible actions—such as buying a card, collecting gems, or reserving a card—by assigning a score to each option using weighted factors. These factors include the prestige points of a card, its effective cost after discounts, the usefulness of its bonus, and how well the action supports the bot’s current goal (for example, targeting a specific card or noble). The bot then selects the action with the highest score. This approach is essentially a structured calculation designed to simulate human-like strategy, not a machine learning AI, because it does not learn or adapt over time but instead follows predefined logic and weights..*

(just calculation)

- **Rule-Based Validation:** *A central logic engine that enforces game rules, such as gem limits (max 10), card reservation limits, and noble visitor requirements.*
- **Data Aggregation Algorithm:** *A process that filters raw gameplay logs to calculate specific features like turn-to-win ratios and gem color popularity across 100+ gameplay sessions.*
- **Event-Driven Interaction:** *Utilizes the Pygame event loop to handle user inputs, screen refreshes, and state updates in real-time.*

4. Statistical Data (Prop Stats)

4.1 Data Features

The game will continuously track player and bot decisions. A single "row" of data will represent either a completed game summary or a specific turn action. To easily exceed the 100-row requirement, data will be gathered through human gameplay and automated "Bot vs. Bot" background simulations. The 5 key features tracked are:

- **Gold Token Usage:** *Records how many gold tokens each bot uses when purchasing cards, showing how much each bot relies on flexible resources and reserve-related strategy.*
- **Total Turns per Match:** *Tracks the total number of turns in each Bot vs. Bot match, measuring overall game pacing and strategy speed.*
- **Gem Color Collection Frequency:** *Logs which gem colors (Ruby, Sapphire, Emerald, Diamond, and Onyx) are collected most frequently by the bots during matches, showing resource preference patterns.*
- **Card Tier Purchased:** *Records the frequency of Tier 1, Tier 2, and Tier 3 cards bought, analyzing whether the bots focus more on short-term engine building or long-term high-value strategies.*
- **Final Score Margin:** *Records the point difference between the winner and the loser at the end of each Bot vs. Bot match, showing how close or one-sided the games are.*

4.2 Data Recording Method

Explain how the game will store statistical data. Will it be saved in a database, CSV file, or another format?

Statistical data will be automatically stored in a **CSV** file.

- **Process:** The `DataLogger` class will append a new row of data to the CSV file at the end of every match (or turn), ensuring data is safely recorded without affecting gameplay performance.

4.3 Data Analysis Report

Outline how you will analyze the recorded data. What statistical measures will you use? How will the analysis be presented (e.g., graphs, tables, charts)?

- The recorded gameplay data will be analyzed to extract meaningful insights about bot behavior and game strategy in Bot vs. Bot matches. The analysis will focus on the five selected features: Gold Token Usage, Total Turns per Match, Gem Color Preference, Card Tier Purchased, and Final Score Margin.
- **Statistical Measures:**
We will use descriptive statistics to summarize the collected data. For numerical features such as total turns per match, gold token usage, and final score margin, we will calculate measures such as the mean, minimum, maximum, and distribution of values. For categorical or frequency-based features such as gem color preference and card tier purchased, we will calculate total counts and relative frequencies to identify the most common patterns. These measures will help show how the bots behave, which resources they prefer, and how competitive the matches are.
- **Presentation:**
The analysis results will be presented using both tables and graphs. A statistical summary table will be used to show key numerical values such as averages and frequencies. Graphical presentation will be created using Python libraries such as Pandas, NumPy, and Matplotlib. The recorded data will first be stored in CSV or JSON

format, then loaded into a Pandas DataFrame for structured processing and analysis.

- *For visualization, different graph types will be used depending on the feature. A pie chart will be used for Gold Token Usage to show the proportion of gold token use between the two bots. A histogram will be used for Total Turns per Match to show the distribution of game lengths. A stacked bar chart will be used for Gem Color Preference to compare the frequency of each gem color collected by the bots. A bar chart will be used for Card Tier Purchased to compare how often Tier 1, Tier 2, and Tier 3 cards are bought. A boxplot will be used for Final Score Margin to show the spread of score differences and whether matches are usually close or one-sided.*
- *These visualizations and statistical summaries will be presented in a dedicated Statistics Dashboard, either generated directly in Pygame or displayed using saved graph images. This approach ensures that both the numerical analysis and the visual presentation are clearly defined, easy to interpret, and reproducible.*

Feature	Why it good to have this data ? What it can be used for ?	How can you obtain 100 values of feature data?	Which variable? (and which class you collect from ?)	How do you display the feature data? (via summarization stat or via graph)
Gem Color Collection Frequency	Reveals which gem colors are preferred by bots, showing resource strategy patterns.	Count how many times each gem color is collected across matches.	From <code>Player</code> / <code>BotPlayer</code> class (token collection)	Stacked bar chart
Card Tier Purchase Frequency	Analyzes whether bots prefer early-game (Tier 1) or late-game (Tier 3) strategies.	Record number of Tier 1, 2, and 3 cards purchased per match.	From <code>Player.cards_owned</code> and <code>Card.level</code> .	Bar chart (Tier 1 vs Tier 2 vs Tier 3)
Final Score Margin	Shows how competitive each match is (close vs dominant games).	Record score difference between bots in each match.	From <code>Game</code> class (final scores).	Boxplot
Gold Token Usage	Shows how much bots rely on gold tokens for flexible purchasing.	Count gold tokens used per match.	From <code>Player</code> / <code>BotPlayer</code> (when spending gold).	Pie chart
Total Turns per Match	hows game pace and efficiency of each strategy.	Record total turns for each match across at least 100 matches.	From <code>Game</code> class (turn counter).	Histogram

Feature Name	Graph objective	Graph type	X-axis	Y-axis:
Gem Color Collection Frequency	To identify which gem colors are collected most often and reveal resource preference patterns of bots.	Stacked bar chart	Gem color (Red, Blue, Green, White, Black)	Total number of tokens collected
Card Tier Purchase Frequency	To analyze whether bots prefer low-tier or high-tier cards, reflecting short-term vs long-term strategies.	Bar chart	Card tier (Tier 1, Tier 2, Tier 3)	Number of cards purchased
Final Score Margin	To measure how competitive matches are by analyzing score differences between bots.	Boxplot	Score margin data set	Score difference (winner score - loser score)
Gold Token Usage	To show the proportion of total gold token usage between Bot A and Bot B, indicating which bot relies more on reserved-card support and flexible resources.	Pie chart	Sections represent Bot A and Bot B	None

Total Turns per Match	To analyze the distribution of match lengths and observe how many turns games usually take	Histogram	Number of turns per match	Frequency (number of matches)
--------------------------	--	-----------	---------------------------------	-------------------------------------

5. Project Planning Timeline

Week	Week	Task
1	8	Proposal submission / Project initiation
2	9	Core OOP & UI Setup (Classes, Pygame display, assets)
3	10	Game Mechanics (Turn system, gem collection, card buying)
4	11	AI Bot Development (Heuristic logic and auto-play)
5	12	Data Logging (CSV setup, simulating 100+ records)
6	13	Analytics & Polish (Stats visualization, debugging, UI improvements)
7	14	Submission week (Draft)

6. Document version

Version: 1.0

Date: 4 March 2025

Editor: Paramee Saejia

Date	Name	Feedback
18/03	Peraya	Major Change needed - Section 4.3: You listed chart types (Pie, Bar, etc.) but didn't explain the Data Processing Pipeline. A useful visualization should help a developer balance the game or understand player strategy. You must specify the X and Y axes for your charts and what insight they provide.

		Minor Suggestions - Section 3.3 Algorithms: You mentioned Heuristic Decision Logic. To make this even clearer for the documentation, specify what you are using.
21/03	Amornrit	Very good. Explain briefly how you would implement “Heuristic Decision Logic (AI)”